

RAGnarok.cpp: A Bare-Metal Retrieval-Augmented Generation Engine in C++17

Aadarsh Kumar Gupta

Department of Computer Science, King's College

Artificial Intelligence & Machine Learning

August 9, 2026

Abstract

Large language models answer questions using only what they learned during training, which means their knowledge is frozen at a cutoff date and cannot include private or domain-specific documents. Retrieval-Augmented Generation (RAG) addresses this by retrieving relevant text at query time and feeding it to the model as context, but almost every RAG implementation available today is built in Python on top of orchestration frameworks like LangChain or LlamaIndex. This project builds a RAG pipeline directly in C++17, without a framework, using the Gemini API for embeddings and generation and MongoDB Atlas for vector storage and retrieval. The system is organized into three components — a Gemini API client, a MongoDB client, and an orchestration engine that ties ingestion and querying together — exposed through a command-line chatbot. Rather than assume C++ is inherently better suited to this task, the project treats that as an open question and measures it: ingestion throughput, retrieval and generation latency (separated from network and API latency), memory footprint, and build reproducibility. The expected contribution is not a faster RAG system, but a documented account of what changes, architecturally and practically, when RAG infrastructure is built at a lower level of abstraction than the ecosystem currently favors.

Introduction

Language models generate text based on patterns learned during training, which means two things follow directly from how they work: they cannot answer questions about anything that happened after their training cutoff, and they have no access to private documents that were never part of their training data. Retraining a model every time new information needs to be incorporated is not practical for most applications, so a different approach has taken hold: retrieve relevant text from an external source at query time and give it to the model as context, then let it generate an answer grounded in that context instead of purely in what it memorized.

Lewis et al. (2020) formalized this as Retrieval-Augmented Generation, combining a retriever over an external document index with a generator that conditions on the retrieved passages. RAG has become a widely adopted architecture for building applications on top of language models, particularly where the domain data is proprietary, frequently updated, or too large to fit in a model's context window. The typical implementation stack looks something like this: documents are split into chunks, each chunk is embedded into a vector using an embedding model, the vectors are stored in a vector database or a vector index attached to a general-purpose database, and at query time the same embedding process is applied to the user's question so that nearest-neighbor search can find semantically related chunks. Almost all of the tooling built around this pattern lives in Python. LangChain, LlamaIndex, Haystack, and similar frameworks provide high-level abstractions over embedding calls, vector stores, and prompt construction, and they have made RAG accessible to a very large number of developers. That accessibility is a genuine strength. But it also means there is comparatively little public documentation of what a RAG pipeline looks like when it is built without those abstractions, in a systems language, with manual control over memory, connection lifetimes, and error propagation. That gap is what this project investigates.

Problem Statement

The problem is not that Python-based RAG tooling is bad. It solves the problem it was built for, and it does so well. The problem is narrower: the current ecosystem gives backend and systems engineers very little to look at if they want to understand what a RAG pipeline actually requires at the

level of HTTP requests, BSON documents, connection pools, and RAI-managed resources, rather than at the level of a framework's `retriever.invoke()` call.

There is a legitimate engineering question underneath the language choice. Does implementing RAG in C++17, using production-grade external services (Gemini for models, MongoDB Atlas for vector search) but no orchestration framework, produce a system that is still correct, maintainable, and reasonably ergonomic to build and extend? Or does the absence of a framework mostly just mean re-implementing boilerplate that Python developers get for free? This matters for anyone doing AI infrastructure work in performance-sensitive or resource-constrained environments — embedded systems, game engines, high-frequency backend services — where pulling in a Python runtime and a large dependency tree is not always an option, and where understanding what the framework is doing under the hood has real value even if you never build the whole thing yourself.

Research Question

Primary research question: To what extent can a modular C++17 implementation provide a correct, reproducible, and resource-efficient RAG pipeline, while maintaining sufficient developer ergonomics for practical AI backend development?

Subquestions:

RQ1 (Feasibility): Can a complete RAG ingestion and query pipeline be implemented in C++17 using production-grade external services (Gemini API, MongoDB Atlas Vector Search) without a dedicated orchestration framework?

RQ2 (Architecture): What architectural and engineering trade-offs arise when RAG is implemented without the abstractions a framework like LangChain would normally provide — and which of those abstractions turn out to be doing real work versus adding a layer of indirection?

RQ3 (Performance): What are the observed latency and memory characteristics of the implementation, and how much of end-to-end latency is attributable to the local application versus external network and API calls?

RQ4 (Ergonomics): What developer-experience costs and benefits come with building this pipeline at a lower level of abstraction — measured in terms of dependency count, boilerplate, build complexity, and time to implement each pipeline stage?

Aim and Objectives

Aim: To design, implement, and evaluate a modular C++17 RAG engine that integrates cloud-based embedding and generation services with vector retrieval, and to document the engineering trade-offs of implementing this class of AI infrastructure without a high-level orchestration framework.

Objectives:

Design a modular three-component architecture (GeminiClient, MongoDBClient, RagEngine) with clearly separated responsibilities and interface boundaries.

Implement a type-safe C++ client for the Gemini embedding and generation APIs, including request/response serialization and error handling.

Implement a MongoDB Atlas client that handles document upserts, embedding storage, and \$vectorSearch queries.

Implement document ingestion (chunking, embedding, storage) and query-time context assembly.

Build a functional, interactive CLI chatbot that exercises the full pipeline end to end.

Establish a reproducible build using CMake 3.20+ and vcpkg manifest mode.

Write automated tests covering the core components, particularly serialization and error paths that don't require live network calls.

Benchmark ingestion throughput, query latency (decomposed by pipeline stage), memory usage, and startup time.

Document the architectural and ergonomic trade-offs observed during implementation, including where a framework's abstractions would have saved meaningful effort.

Significance

Academic. The project applies the RAG architecture described by Lewis et al. to a concrete implementation and requires understanding retrieval, embedding spaces, and grounded generation well enough to build each piece rather than call a library function for it.

Engineering. Building the pipeline without a framework forces explicit decisions about resource lifetime, connection management, error propagation across three external boundaries (HTTP, MongoDB driver, CLI), and configuration and secret handling.

Professional. The finished system demonstrates modern C++ practice (RAII, smart pointers, const-correctness), integration with cloud AI and database services, and a build system suitable for team development.

Literature and Technical Foundation

Academic literature. The foundational reference is Lewis, P., Perez, E., Piktus, A., et al. (2020), "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS 2020. Karpukhin et al. (2020), "Dense Passage Retrieval for Open-Domain Question Answering," EMNLP 2020, is relevant background for understanding dense embedding-based retrieval. For evaluation methodology, Es, S. et al. (2023), "RAGAS: Automated Evaluation of Retrieval Augmented Generation," is a useful reference.

Technical documentation. Google's Gemini API documentation; MongoDB's Atlas Vector Search documentation; the ISO C++17 standard; and Lospinoso, J. (2019), C++ Crash Course, for modern C++ idioms.

Proposed System Architecture

Ingestion Pipeline

Document → Chunking → Gemini Embedding API → Embedding Vector → MongoDB Atlas upsert → Stored Document + Metadata + Vector

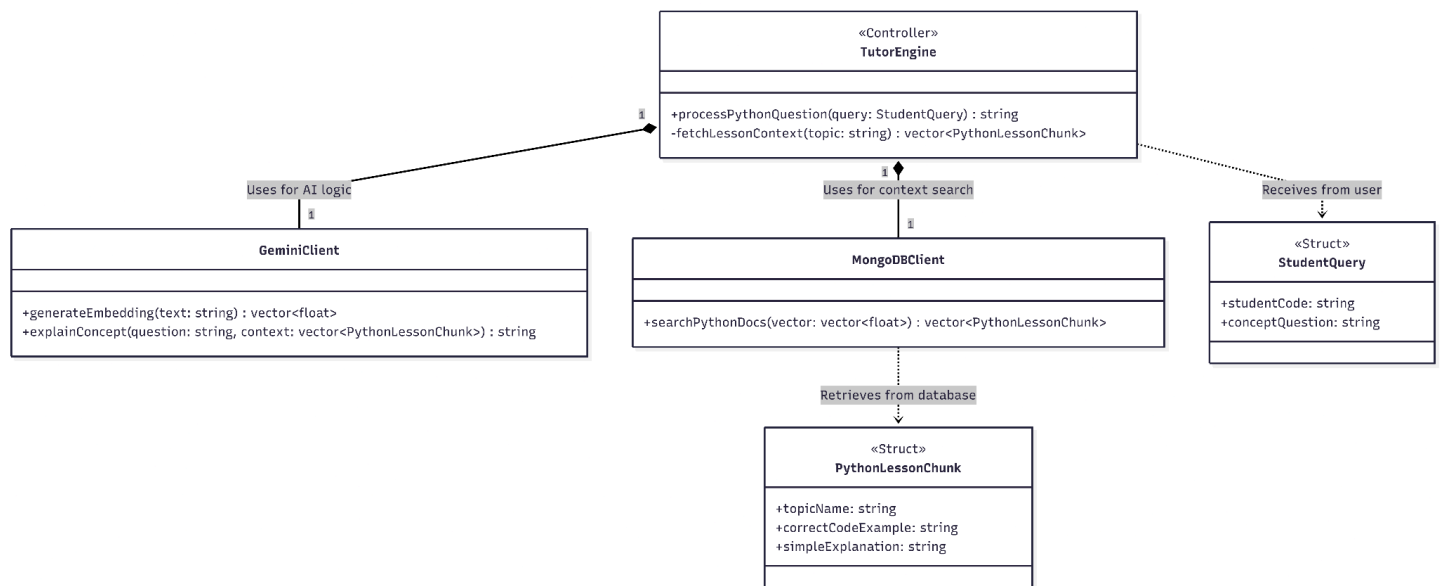
Query Pipeline

User Query → Gemini Embedding API → Query Vector → MongoDB Atlas \$vectorSearch → Top-K Chunks → Context Assembly → Gemini Generation API → Grounded Response

GeminiClient: Owns interaction with the Gemini API: constructing requests, serializing to JSON with nlohmann/json, sending over HTTP via cpr, and parsing responses.

MongoDBClient: Owns the connection to MongoDB Atlas via the mongocxx driver, BSON serialization, and execution of \$vectorSearch queries.

RagEngine: Orchestrates components. Handles chunking strategy, context assembly, and error translation.



Software Engineering Design

RAII and smart pointers: unique_ptr for exclusive ownership (e.g., connection pools); raw references for non-owning access.

Const-correctness: Read-only operations marked const for reasoning about mutation.

Interface boundaries: Headers in include/ragnarok/ define public interfaces; implementation confined to .cpp files.

Error handling: Exceptions for unrecoverable errors; return values for expected failure modes (e.g., retrievable API calls).

Configuration: API keys loaded from environment variables, never hardcoded.

Dependency management: vcpkg manifest mode pins third-party dependencies (cpr, nlohmann/json, mongocxx).

Security and Reliability

In scope:

Environment variable loading for secrets.

Validation of all external API responses.

Timeouts on network calls.

Structural separation of context and instructions in prompts.

Out of scope:

- Advanced backoff strategies.
- Comprehensive prompt-injection defenses.
- Multi-user access control.

Evaluation Methodology

- Functional evaluation:** Verify successful ingestion, storage, retrieval, and full query round-trip.
- Correctness evaluation:** Use a 10–20 document benchmark corpus with verifiable grounded answers and unanswerable questions.
- Performance evaluation:** Decomposed latency (Total = preprocessing + Embedding + Retrieval + Context Assembly + Generation).
- Developer-experience evaluation:** Qualitative assessment of boilerplate, dependency count, and implementation effort.

Success Criteria

Tier	Criteria
Must have	Complete ingestion/query pipeline; Gemini/MongoDB Atlas integration; functional CLI; reproducible CMake/vcpkg build; README instructions.
Should have	Automated unit tests; structured logging; externalized configuration; benchmark harness with latency breakdown.
Could have	Streaming CLI output; metadata filtering; support for PDF/Markdown; conversation history; retrieval evaluation dashboard.

Implementation Timeline

Day	Focus
1	Project scaffolding: CMake, vcpkg, repository structure, environment loading.
2	GeminiClient: HTTP layer, embedding endpoint, response parsing.
3	MongoDBClient: Atlas connection, schema design, document upsert.
4	Vector retrieval: Atlas index config, \$vectorSearch implementation.
5	Generation: Prompt construction, context assembly, generation API integration.
6	End-to-end integration: RagEngine orchestration, CLI interface, error handling.
7	Testing, benchmarking (latency/memory), and final documentation.

Risk Analysis

Risk	Probability	Impact	Mitigation
Gemini API changes	Low ▾	Medium ▾	Isolate API interaction inside GeminiClient.

Risk	Probability	Impact	Mitigation
API rate limiting	Medium ▾	Medium ▾	Implement retry-with-backoff; batch embedding calls.
mongocxx build complexity	Medium ▾	High ▾	Verify build via vcpkg on Day 1.
Vector dimension mismatch	Low ▾	High ▾	Confirm dimensionality from documentation before indexing.
7-day time limit	High ▾	High ▾	Strict adherence to "Must Have" criteria; cut features early.

Limitations

- It does not prove C++ is generally faster than Python.
- A small benchmark corpus (10–20 docs) limits scale claims.
- It does not solve LLM hallucination or prompt injection.
- Single-user CLI prototype only.

Expected Outcomes and Deliverables

- Functional Ragnarok CLI binary.
- Buildable C++17 source code and CMake build system.
- vcpkg manifest with pinned dependencies.
- Working on Gemini and MongoDB Atlas integrations.
- Benchmark results and architecture documentation.

Expected Contribution

This project contributes a concrete, documented account of a RAG pipeline built from lower-level pieces. It surfaces the trade-offs of working without a framework, providing data on where abstractions save effort and what the resulting resource profile looks like for systems-minded developers.

Conclusion

RAGnarok.cpp asks what the standard RAG architecture looks like when built directly in C++17 against production services. By avoiding high-level frameworks, the project exposes the underlying systems engineering requirements—resource management, API serialization, and orchestration—evaluating them through empirical measurement rather than assumption.

References

- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv preprint arXiv:2309.15217.
- Google. Gemini API Documentation. Google AI for Developers.
- International Organization for Standardization. (2017). ISO/IEC 14882:2017 — Programming languages — C++.
- Karpukhin, V., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. Proceedings of EMNLP 2020.
- Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 33.
- Lospinoso, J. (2019). C++ Crash Course: A Fast-Paced Introduction. No Starch Press.
- MongoDB, Inc. Atlas Vector Search Documentation.